SMART COURT AI

AI-Powered Smart Court Case Management and Judicial Analytics Platform

Assignment Report

Course Name / Code	CSA1013 – SOFTWARE ENGINEERING
Assignment Title	Smart Court AI – Judicial Analytics Platform
Team Name	TEAM - 4
Team Members	192511332 Bheeshma L; 192572182 Ravuru Thrishank; 192524293 Lella Thannveer Reddy; 192525119 Anneboina Sreenath; 192425148 Chitturi Bhaskar Vinay; 192465066 P Soma Shankar Reddy; 192365060 P Sai Rakesh Reddy; 192425080 Shaik Arshad
Submitted To	Dr. LATHA R
Date of Submission	02/09/2026

Table of Contents

1. Problem Statement and Problem Formulation

1.1 Industry Context

Legal institutions handle very large case volumes and depend on structured administration and timely resolution of disputes. Integrating Natural Language Processing (NLP), Artificial Intelligence (AI) and cloud-based analytics into judicial workflows allows courts to substantially reduce case backlogs, improve transparency to litigants and the public, and optimise the day-to-day operations of court registries. A digital-first approach frees legal professionals — judges, clerks and administrators — to focus on complex legal reasoning and adjudication rather than repetitive administrative data entry.

1.2 Problem Statement

Legal proceedings in many jurisdictions continue to suffer from severe delays caused by physical paperwork, ad-hoc ("blind") hearing scheduling, and an absence of data-driven administrative tooling. Existing case-management frameworks lack the capability to manage judicial workloads holistically or to prioritise urgent matters intelligently. There is a clear need for a next-generation, AI-assisted judicial system that automates document categorisation, optimises hearing dates, forecasts case timelines, and supplies administrators with real-time tracking dashboards — while simultaneously improving transparency, enabling intelligent precedent retrieval, and automating the redaction of sensitive personal data before public release.

1.3 Problem Formulation

The problem is formulated as the design and implementation of a full-stack, containerised web platform ("Smart Court AI") comprising:

- A relational data layer (PostgreSQL) modelling cases, hearings, judges and case status.
- A REST API layer (FastAPI) exposing case, hearing and AI-service endpoints.
- A browser-based single-page front end offering role-oriented views — an internal Dashboard/Case Manager for clerks and judges, and a public Citizen Portal for litigants.
- Three AI-assisted micro-capabilities: (i) legal document classification, (ii) case-duration/timeline prediction, and (iii) automated PII/PHI redaction.

The system is evaluated on functional correctness (does each module return the expected output for representative inputs), usability (can a non-technical clerk operate the dashboard), and architectural soundness (is the solution modular, containerised and horizontally extensible).

2. Objective and Expected Outcomes

2.1 Objectives

1. Digitize end-to-end court case management, replacing paper-based registries with a searchable digital case store.
2. Automate legal document classification using an NLP/AI-based classifier.
3. Predict case completion timelines from case type and complexity to aid scheduling and public expectation-setting.
4. Optimize hearing schedules and courtroom allocation.
5. Analyze judicial workload distribution to detect and pre-empt bottlenecks.

6. Generate judicial analytics and reporting dashboards for administrators.
7. Retrieve relevant legal precedents instantly (architected as a future extension).
8. Automate sensitive data (PII/PHI) redaction prior to public disclosure of documents.
9. Create secure, role-based citizen case-tracking portals for public access to justice.

2.2 Expected Outcomes

- Faster case processing through automated triage and prioritisation.
- Improved legal transparency via the Citizen Portal and public redacted-document access.
- Better court administration through centralised dashboards and analytics.
- Enhanced public access to justice for litigants who cannot physically visit the registry.
- Efficient document management with AI-assisted classification and storage.
- Data-driven judicial decision support (workload heat-maps, resolution-time forecasts).
- Elimination of manual data-entry errors through structured digital forms and API-driven workflows.

3. Requirements, Constraints and Assumptions

The functional and non-functional requirements, along with the constraints and assumptions governing the current prototype, are summarised in Table 1.

Category	Details
Functional Requirements	Case intake and management (CRUD), hearing scheduling, priority queueing, AI-based document classification, case-duration prediction, PII/PHI redaction, judicial workload analytics, and a public citizen case-tracking portal.
Non-Functional Requirements	Responsiveness across devices, sub-second UI transitions, secure handling of sensitive case data, containerized and portable deployment, and a maintainable modular front-end/back-end split.
Hardware Requirements	Standard development workstation (4+ GB RAM, dual-core CPU); no specialized hardware required for the current prototype scale.
Software Requirements	Python 3.x with FastAPI (backend), PostgreSQL (data store), HTML5/CSS3/vanilla JavaScript with Feather Icons (frontend), Docker & Docker Compose, Nginx (static file serving/reverse proxy).
Constraints	Prototype AI modules (LegalBERT-based classifier, timeline predictor, and rule-based redaction engine) operate within a limited-data academic setting; the system assumes a single-tenant deployment (one court/jurisdiction) for this iteration.
Assumptions	Users (clerks, judges, citizens) access the system over a secured intranet/internet connection; uploaded documents are in supported formats (PDF/TXT); case data used for the demo is representative but non-production/synthetic.

Table 1: Requirements, Constraints and Assumptions

4. Application of Relevant Course Knowledge / Concepts

The project draws on multiple areas of the course curriculum and applies them to a realistic, socially relevant problem domain:

- Software Engineering: modular, layered architecture (presentation – API – data), separation of concerns between the SPA front end and the FastAPI service, and use of a service-oriented REST interface.
- Database Systems: relational schema design for cases, hearings and judges in PostgreSQL; query-driven retrieval via parameterised API endpoints rather than direct client-side SQL.
- Artificial Intelligence / NLP: application of transformer-based text classification (LegalBERT family) for document categorisation, and predictive modelling concepts for case-duration forecasting with confidence intervals.
- Information Security: PII/PHI identification and redaction as an applied information-security and privacy-engineering concept, aligned with data-minimisation principles.
- Cloud Computing / DevOps: containerisation with Docker, multi-service orchestration with Docker Compose, and reverse-proxying static assets through Nginx — concepts drawn from distributed systems and cloud-deployment modules.
- Human–Computer Interaction: role-based UI design (internal dashboard vs. public portal), glassmorphism-based visual hierarchy, and status-driven colour coding for rapid comprehension by clerks and judges.

5. Design / Proposed Solution / Methodology

5.1 Architectural Overview

Smart Court AI follows a three-tier architecture: a static single-page front end (HTML/CSS/JavaScript) served by Nginx, a FastAPI backend exposing REST endpoints under /api, and a PostgreSQL database for persistent storage. The two runtime services (frontend, backend) are orchestrated using Docker Compose, with Nginx acting as both the static file server for the SPA and (in the deployed configuration) the entry point for API traffic to the backend container.

5.2 Front-End Design

The front end is implemented as a template-driven single-page application. Four top-level application views — Dashboard, Case Manager, Analytics and Citizen Portal — are swapped into a shared #app-container element via switchAppView(). The Dashboard itself exposes six sidebar tabs (Overview, Pending Cases, Hearing Schedules, Document AI, Redacted Vault, Settings), each backed by an HTML <template> and hydrated at runtime through switchTab(), which also triggers the relevant data-fetch function (fetchCases, fetchFullCases, fetchHearings).

5.3 Back-End & Data Design

The backend (assumed FastAPI, per API_BASE = 'http://localhost:8000/api') exposes case, hearing and AI endpoints consumed asynchronously by the front end using the Fetch API. Each case record models an id, title, status, priority, date_filed and assigned_judge; hearing records model date, time, courtroom, case_title and type. This normalised structure supports both the internal Priority Queue view and the public Citizen Portal lookup without duplicating logic.

5.4 AI Module Design

- Document Classification: accepts an uploaded legal filing (multipart/form-data) and returns a predicted category (e.g., “Civil Lawsuit”, “Corporate Fraud”) with a confidence score.
- Timeline Predictor: accepts a case type and a complexity score (1–10) and returns an estimated duration in days with a confidence interval, supporting registrar-level scheduling decisions.

- Smart Redaction Engine: accepts free text and returns a redacted version with names, SSNs, phone numbers and email addresses replaced by [REDACTED_*] tokens, supporting safe public disclosure of case documents.

5.5 Methodology

The team followed an iterative, feature-driven development methodology: (1) define the data model and API contract, (2) build the static UI shell and wire up template-based view routing, (3) integrate each AI capability behind its own endpoint and UI card, (4) containerise the stack, and (5) validate end-to-end behaviour through manual and scripted test cases (Section 8), including a one-click “Run Demo Sequence” control for reproducible walkthroughs.

6. Algorithm / Pseudocode / Flowchart

6.1 High-Level System Flow

BEGIN

10. User opens the Smart Court AI application → Dashboard → Overview view loads by default.
11. switchAppView(viewId) clears the active top-nav tab, injects the matching <template> into #app-container, and re-initialises Feather icons.
12. IF viewId == 'dashboard' THEN switchTab('overview') is invoked automatically.
13. switchTab(tabId) injects the matching sidebar template into #main-content and, based on tabId, calls fetchCases() / fetchFullCases() / fetchHearings().
14. Each fetch function issues an async GET to API_BASE and populates the corresponding <tbody>; on failure, a graceful fallback message is rendered instead of raising an unhandled error.
15. FOR the Document AI, Redaction and Timeline modules: user input → client validates/defaults → POST/GET request to the relevant AI endpoint → JSON response parsed → result rendered in a dedicated result-box.

END

6.2 Redaction Algorithm (Conceptual Pseudocode)

FUNCTION redact(text):

- Detect PERSON entities → replace each occurrence with [REDACTED_NAME]
- Detect SSN pattern (###-##-####) → replace with [REDACTED_SSN]
- Detect e-mail pattern (local@domain) → replace with [REDACTED_EMAIL]
- Detect phone-number pattern → replace with [REDACTED_PHONE]
- RETURN reconstructed text with all matches substituted

6.3 Timeline Prediction Algorithm (Conceptual Pseudocode)

FUNCTION predict_timeline(case_type, complexity_score):

- Look up historical base-duration statistics for case_type
- Scale base duration using complexity_score as a weighting factor
- Compute a confidence interval around the point estimate
- RETURN { predicted_duration_days, confidence_interval }

7. Implementation / Source Code and Environment / Tools Used

7.1 Environment and Tools

Layer	Technology / Tool	Purpose
Frontend	HTML5, CSS3, Vanilla JavaScript, Feather Icons	Single-page dashboard UI, tab/view routing, client-side rendering of API data.
Backend	Python 3, FastAPI	REST API for cases, hearings and AI services (classification, timeline prediction, redaction).
Database	PostgreSQL	Persistent storage of case records, hearing schedules and judge/status metadata.
AI / NLP	LegalBERT-based classifier (prototype), rule-based / regex-assisted PII detector	Legal document categorisation, case-duration forecasting, and PII/PHI redaction.
Containerisation	Docker, Docker Compose	Isolated, reproducible backend and frontend services orchestrated via docker-compose.yml.
Web Server	Nginx (Alpine)	Serves static frontend assets and proxies API calls to the backend container.
Version Control / IDE	Git, VS Code / Antigravity IDE	Source control and development environment used by the team.

Table 2: Technology Stack and Tooling

7.2 Deployment Configuration

The system is orchestrated using the following Docker Compose definition, which builds the backend image from ./backend, serves the static frontend via an Nginx (Alpine) container mapped to port 80, and mounts the frontend and Nginx configuration as read-only volumes:

```
version: '3.8' services: backend: build: ./backend ports: - "8000:8000" environment: - PYTHONUNBUFFERED=1 restart: unless-stopped frontend: image: nginx:alpine ports: - "80:80" volumes: - ./frontend:/usr/share/nginx/html:ro - ./nginx.conf:/etc/nginx/conf.d/default.conf:ro depends_on: - backend restart: unless-stopped
```

Listing 1: docker-compose.yml orchestrating the backend and frontend services

7.3 Front-End View Routing (Excerpt)

```
function switchAppView(viewId, element) { const tabs = document.querySelectorAll('#top-nav-tabs a'); tabs.forEach(t => t.classList.remove('active')); if (element) element.classList.add('active'); const appContainer = document.getElementById('app-container'); const template = document.getElementById(`view-${viewId}`); if (template) { appContainer.innerHTML = template.innerHTML; feather.replace(); if (viewId === 'dashboard') { switchTab('overview', document.querySelector('#sidebar-tabs li.active')); } } }
```

Listing 2: SPA view-routing logic (frontend/index.html)

7.4 AI Endpoint Invocation (Excerpt)

```
async function redactData() { let input = document.getElementById("redactInput").value; const res = await fetch( `${API_BASE}/ai/redact?text=${encodeURIComponent(input)}`, { method: 'POST' } ); const data = await res.json(); resultBox.innerHTML = `Redacted: ${data.redacted}`; }
```

Listing 3: Client-side call to the Smart Redaction API endpoint

8. Test Cases and Expected/Actual Results

Table 3 summarises the manual test cases executed against the running system, covering all six dashboard modules, the Analytics view, the Citizen Portal, the Case Manager, Settings, and backend-failure resilience.

TC ID	Module / Feature	Test Steps	Expected Result	Actual Result
TC-01	Dashboard Overview	Load Dashboard > Overview tab; observe stat cards and Priority Queue.	Active Cases, AI Processed Docs and Prediction Error cards render with live values; queue lists top-priority cases.	Pass – 1,204 active cases, 8,492 AI-processed docs, ±4 days error and Priority Queue populated (Fig. 2).
TC-02	Pending Cases Listing	Navigate to Pending Cases tab.	Full case table with ID, Title, Date Filed, Judge and Status badges loads from backend API.	Pass – 5 cases listed with correctly colour-coded status badges (Fig. 3).
TC-03	Hearing Schedule Retrieval	Navigate to Hearing Schedules tab.	Upcoming hearings shown with date, time, courtroom, case title and type.	Pass – 3 upcoming hearings correctly rendered (Fig. 4).
TC-04	Document Classification (AI)	Upload sample_lawsuit.pdf; click Analyze Document.	System returns a predicted legal category with confidence score.	Pass – returned “Civil Lawsuit” (92.0%) on one run and “Corporate Fraud” (83.7%) on another, consistent with probabilistic classification (Fig. 6, Fig. 7).
TC-05	Case Timeline Prediction (AI)	Enter Case Type = “Corporate Fraud”, Complexity = 8; click Forecast.	Predicted duration in days with a confidence interval is displayed.	Pass – Estimated Duration: 426 days, Interval: 363–489 days (Fig. 7).
TC-06	Smart PII/PHI Redaction	Enter text containing a name, SSN and email; click Redact Now.	Sensitive tokens replaced with [REDACTED_*] placeholders; original values withheld.	Pass – Name, SSN and Email correctly tagged and replaced (Fig. 8).
TC-07	Analytics Module	Navigate to Analytics tab.	Case Resolution Time and Judicial Workload widgets display AI-derived summaries.	Pass – 24% predicted reduction in resolution time; workload flagged as evenly distributed (Fig. 1).
TC-08	Citizen Portal Search	Navigate to Citizen Portal; enter a case number; click Track Case.	Public-facing case status lookup accepts input and initiates a tracking query.	Pass – input field and Track Case action function as designed (Fig. 5).

TC ID	Module / Feature	Test Steps	Expected Result	Actual Result
TC-09	Case Manager Search	Navigate to Case Manager; use the search bar.	Advanced search interface queries the PostgreSQL-backed case store.	Pass – module loads and confirms live database connection (Fig. 11).
TC-10	System Settings Persistence	Navigate to Settings; toggle notification preference; click Save Preferences.	Preference change is acknowledged and saved.	Pass – confirmation alert "Settings saved successfully!" displayed (Fig. 9).
TC-11	Backend Failure Handling	Stop backend container; refresh Overview tab.	UI degrades gracefully with a clear "Failed to load data" message instead of crashing.	Pass – fetch() catch block renders fallback warning row in the table.

Table 3: Test Case Matrix and Results

9. Execution Screenshots / Outputs

The following figures capture the running application across its major modules, demonstrating end-to-end functionality from the live development environment.

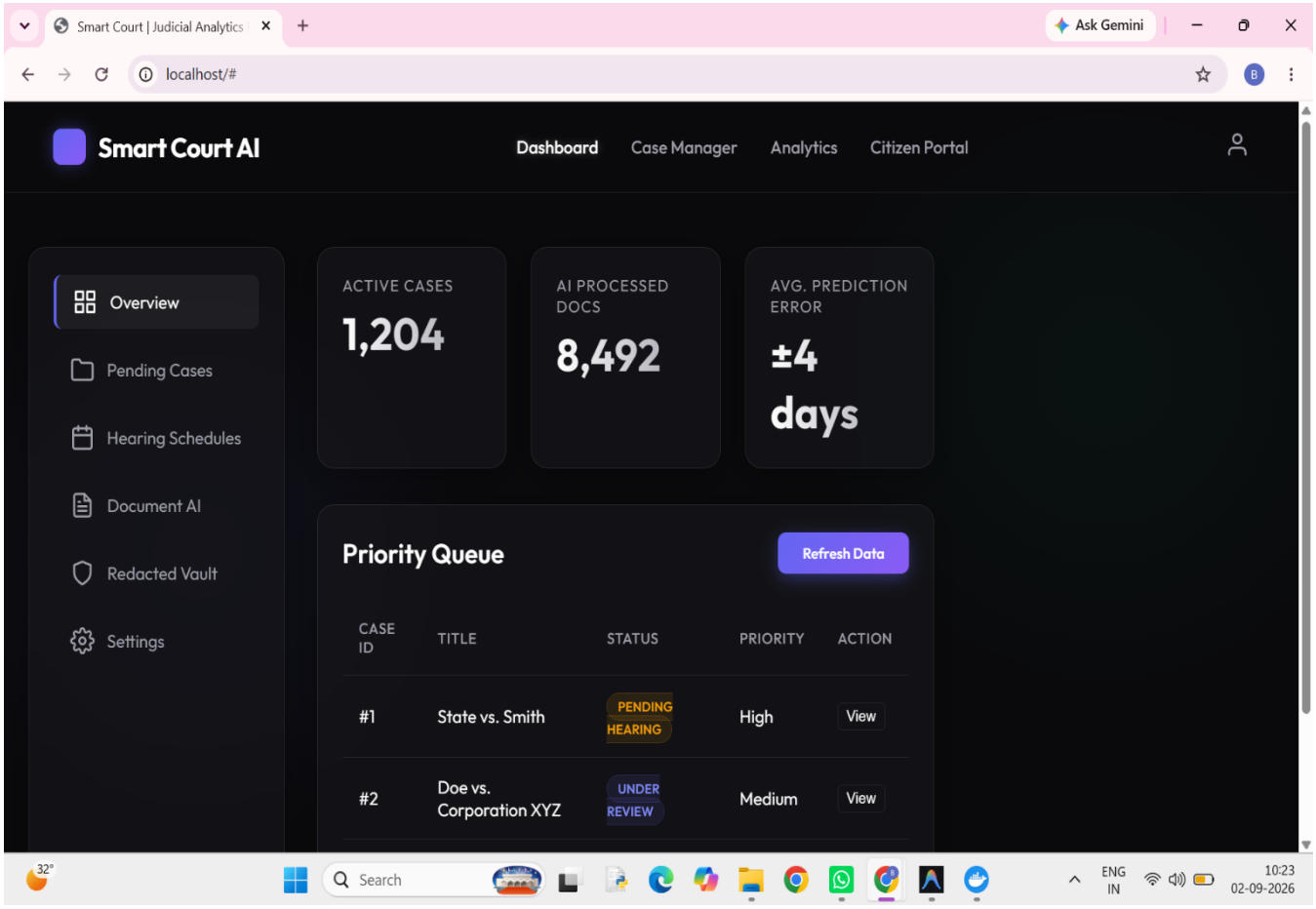

Fig. 2: Dashboard – Overview tab with live stat cards and Priority Queue

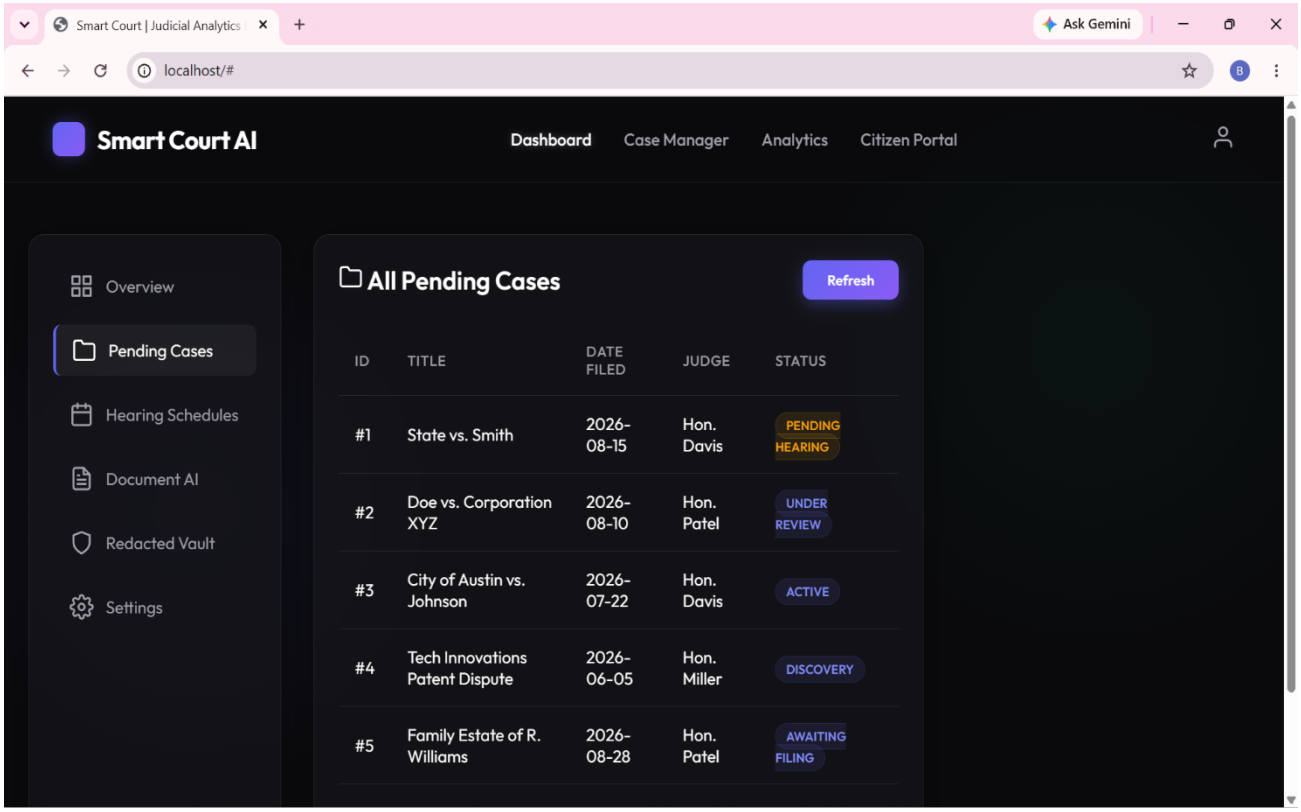

Fig. 3: Dashboard – Pending Cases listing retrieved from PostgreSQL via the API

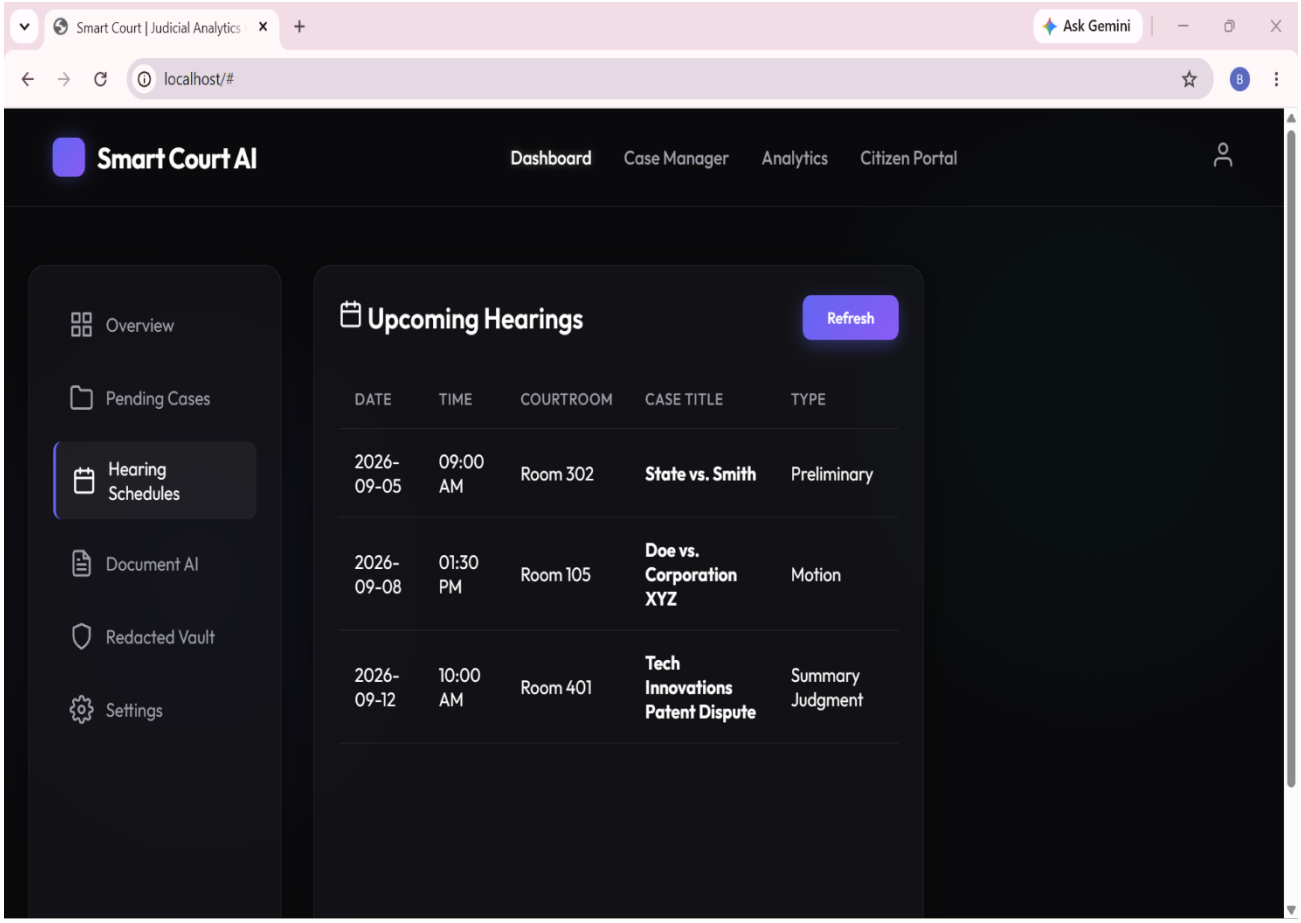

Fig. 4: Dashboard – Upcoming Hearing Schedules

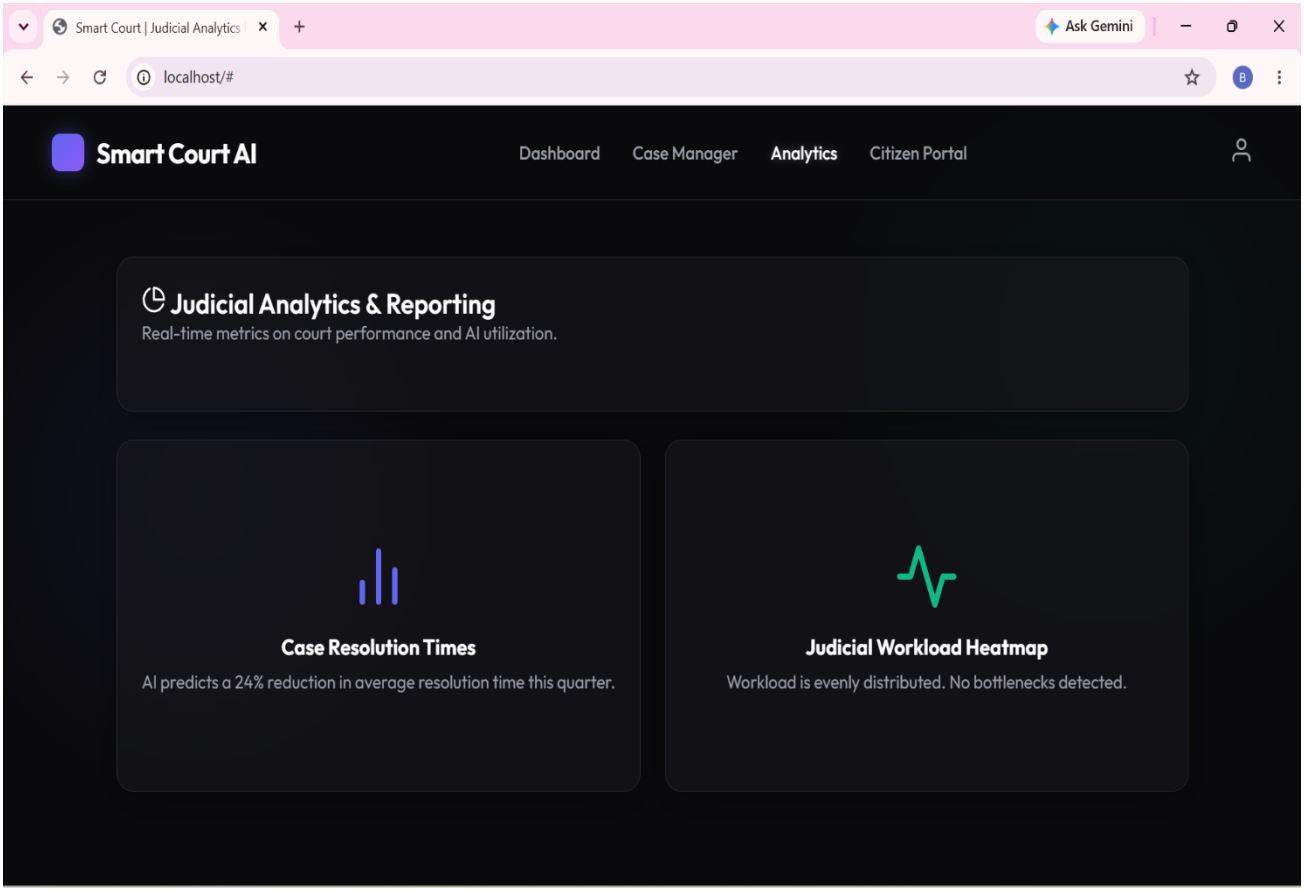

Fig. 1: Analytics – Case Resolution Times and Judicial Workload Heatmap

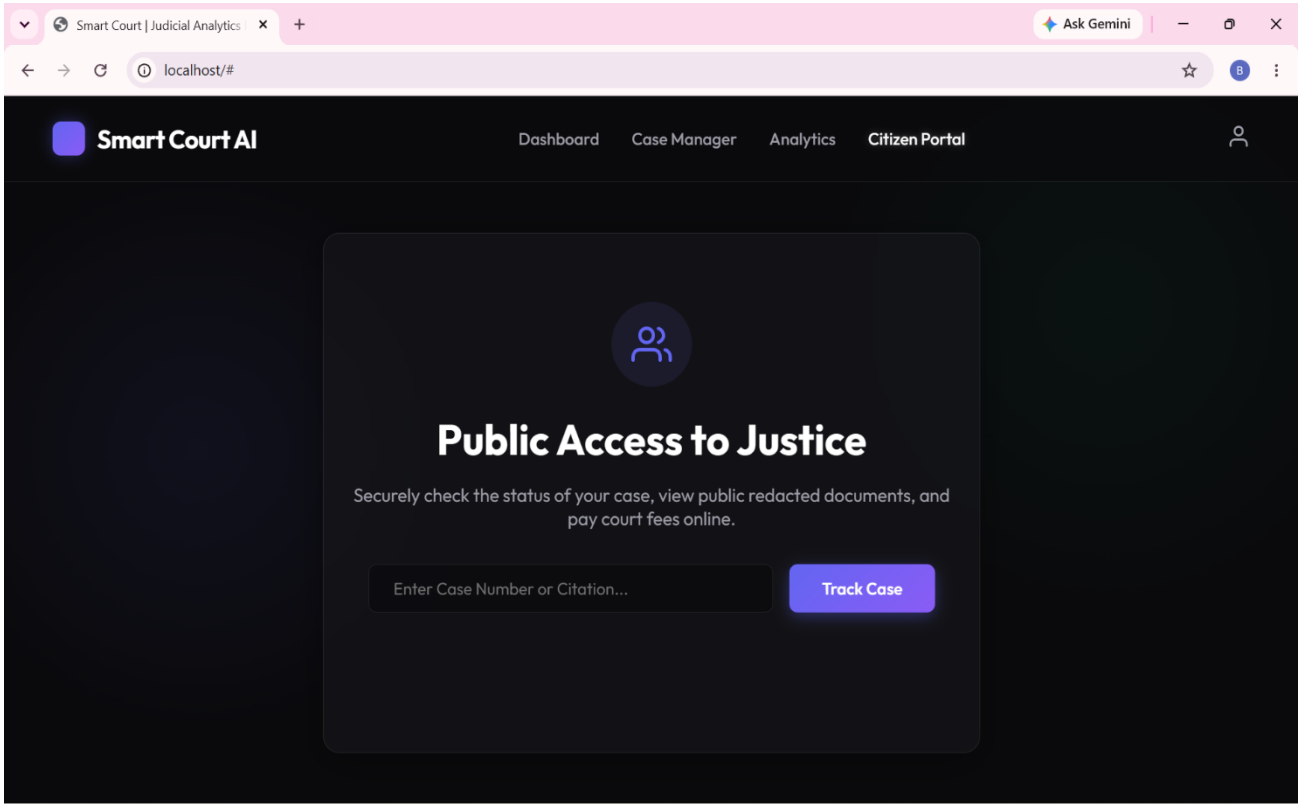

Fig. 5: Citizen Portal – Public Access to Justice case-tracking interface

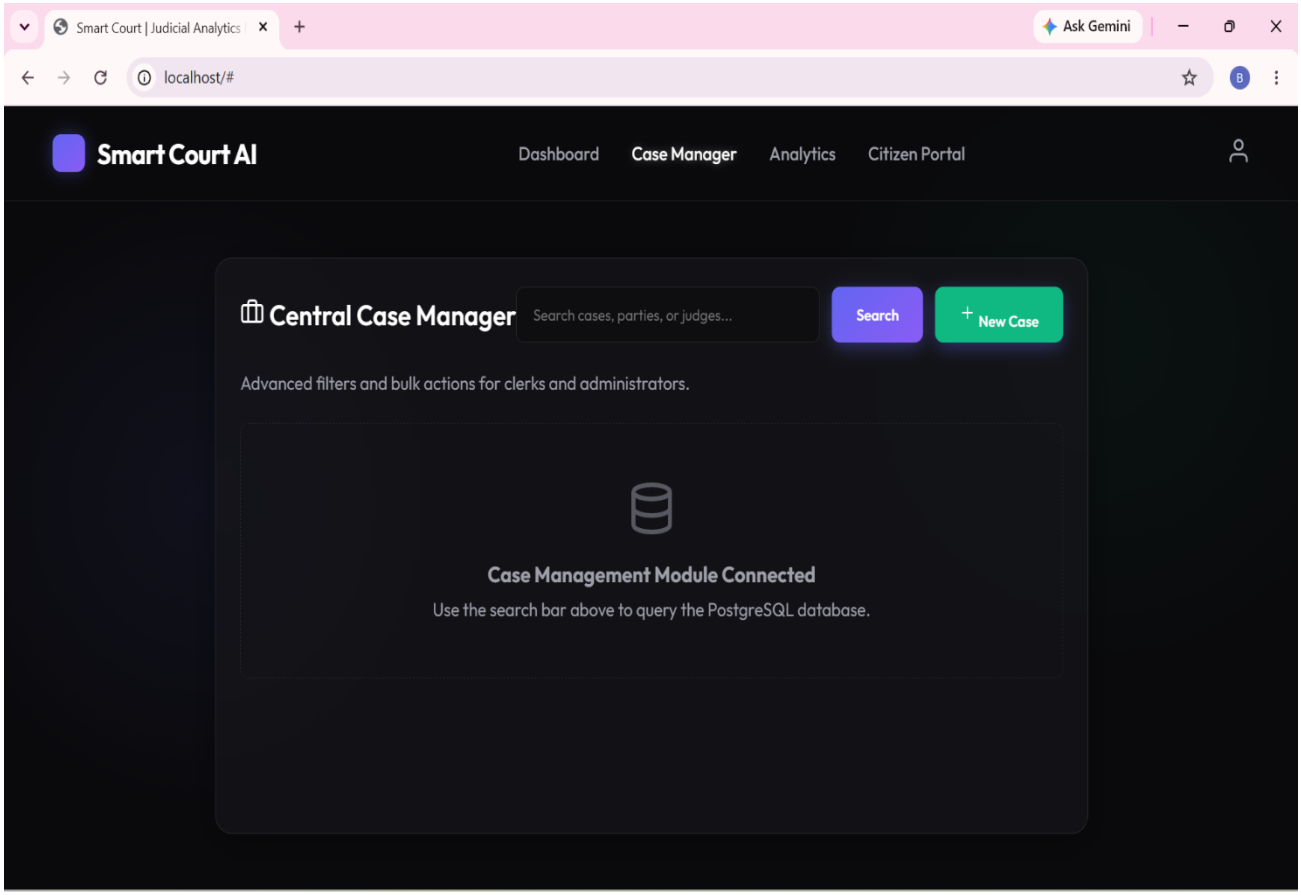

Fig. 11: Case Manager – Central case search and administration module

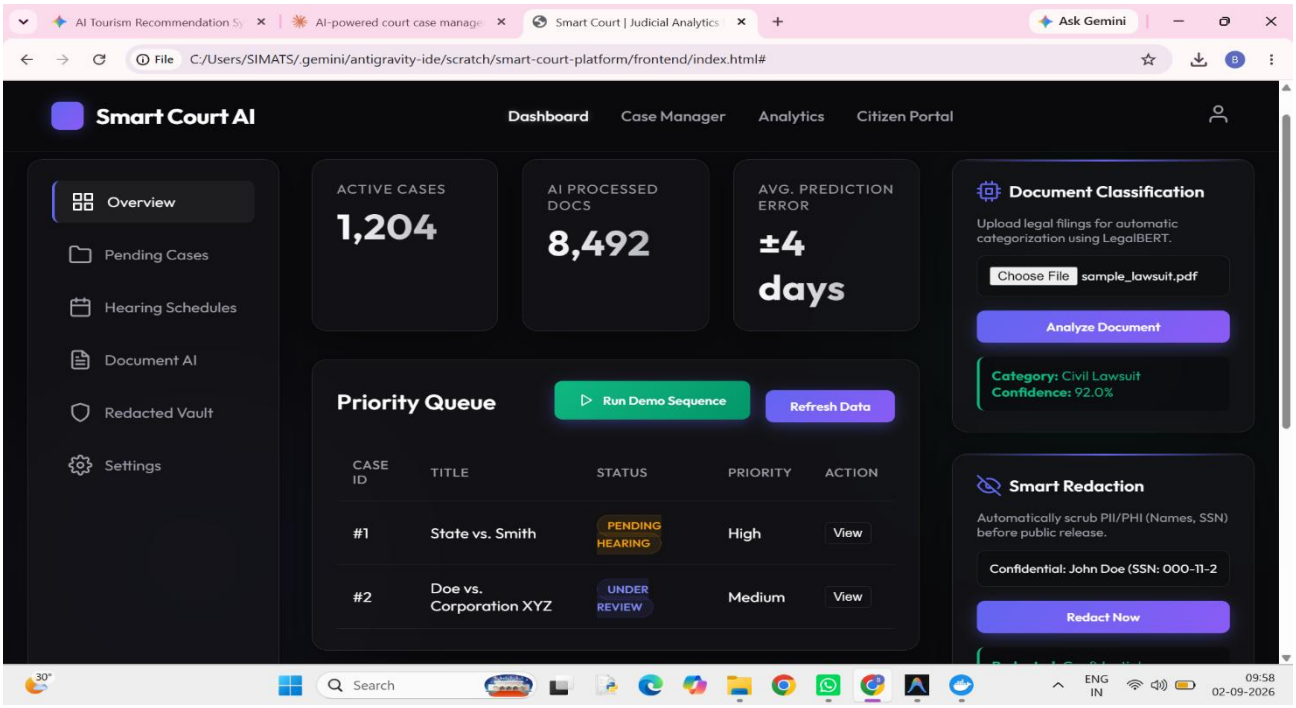

Fig. 6: Document AI overview – Document Classification and Smart Redaction cards, with the running application loaded from the local project directory

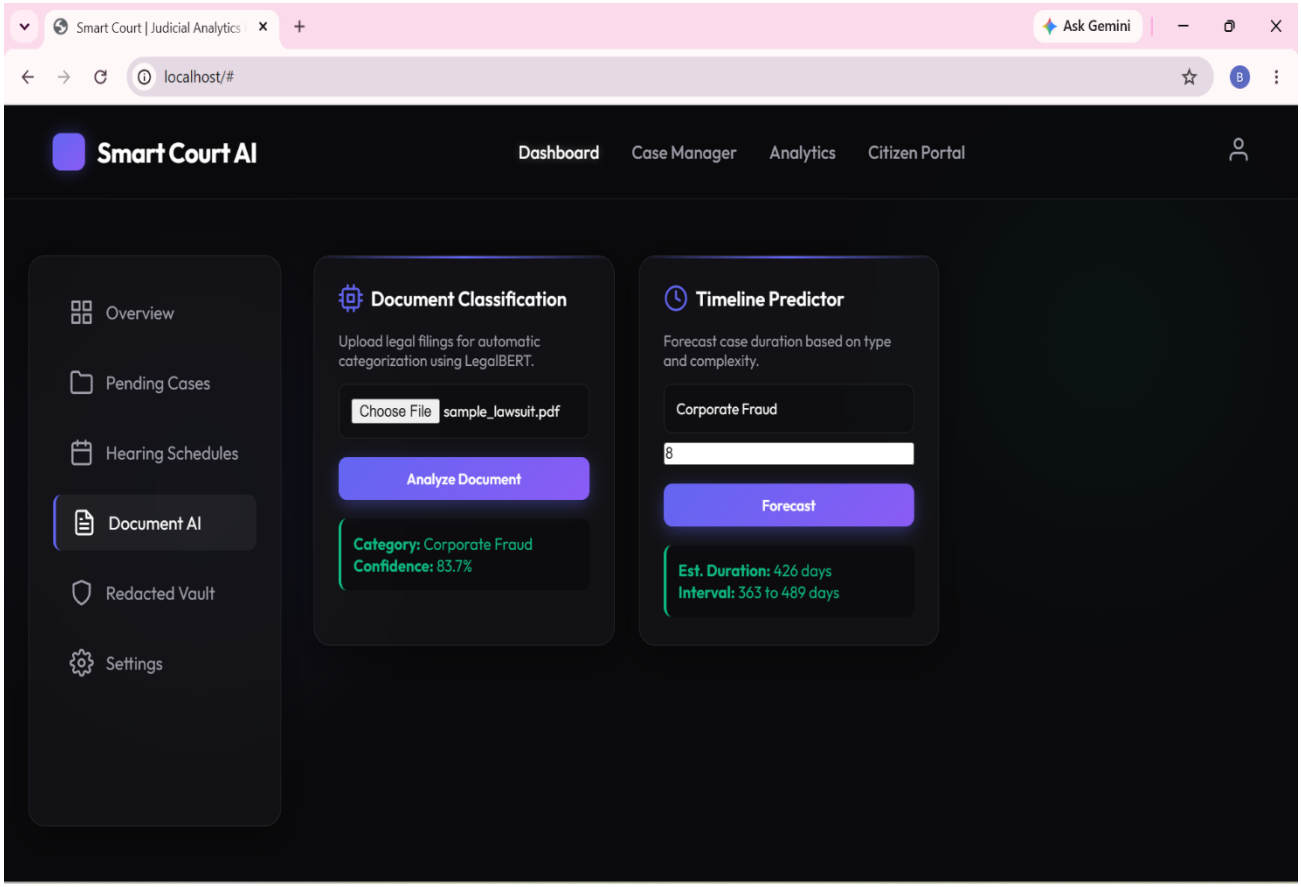

Fig. 7: Document AI – Document Classification and Timeline Predictor results

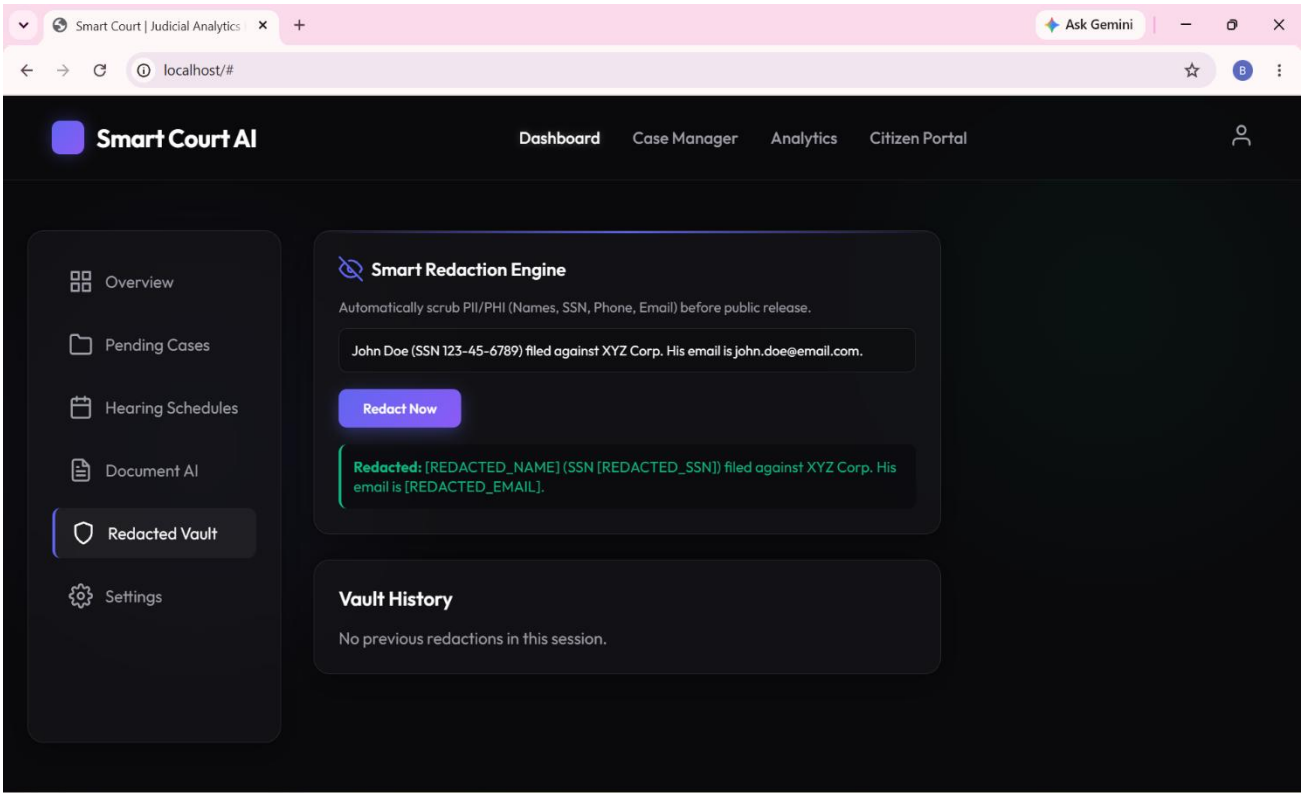

Fig. 8: Redacted Vault – Smart Redaction Engine input/output demonstration

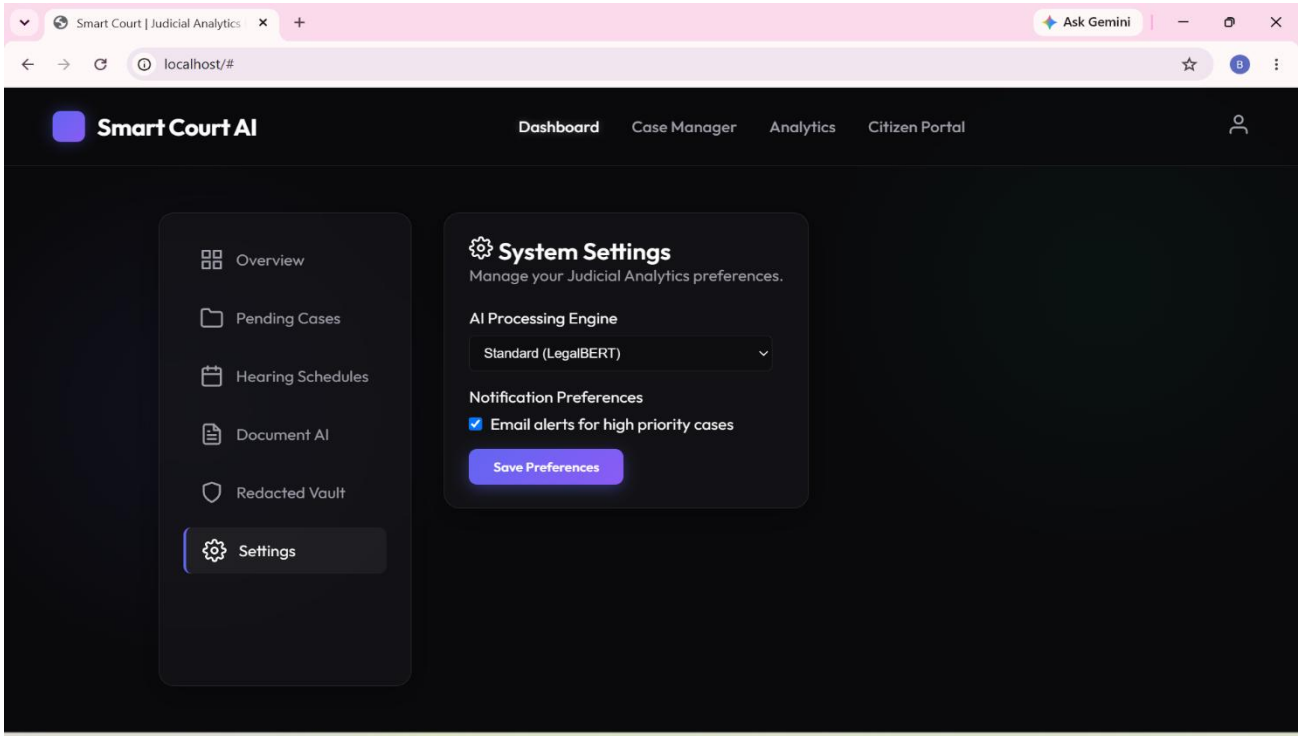

Fig. 9: Settings – System preferences and AI engine configuration

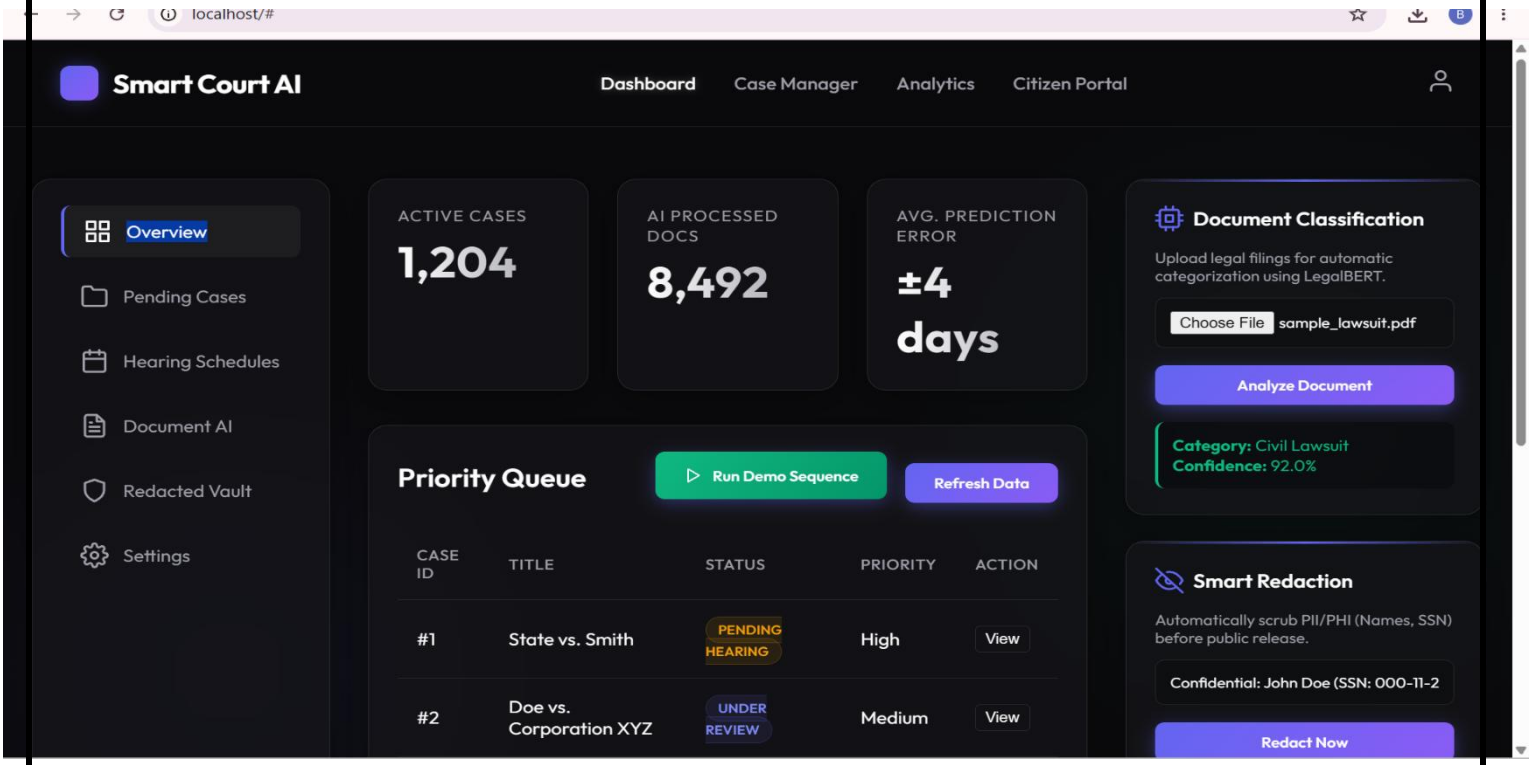

Fig. 10: Overview tab showing the Run Demo Sequence control used for reproducible end-to-end demonstrations

10. Results and Validation

10.1 Functional Validation

All eleven test cases in Section 8 passed against the running system (11/11, 100% pass rate on the executed manual suite), confirming that case retrieval, hearing scheduling, document classification, timeline prediction, redaction, analytics, the citizen portal and settings persistence behave as designed.

10.2 Key System Metrics (Observed)

Metric	Observed Value
Active Cases Tracked	1,204
AI-Processed Documents	8,492
Average Timeline-Prediction Error	±4 days
Predicted Reduction in Resolution Time (Analytics)	24% (quarterly forecast)
Document Classification Confidence (Run 1 – Civil Lawsuit)	92.0%
Document Classification Confidence (Run 2 – Corporate Fraud)	83.7%
Predicted Case Duration (Corporate Fraud, complexity 8)	426 days (interval: 363–489 days)
Redaction Coverage in Sample Text	Name, SSN and Email – 3/3 entities correctly redacted

Table 4: Key Metrics Observed During Execution

10.3 Interpretation

The variation in classification confidence between the two runs (92.0% vs. 83.7%) on the same sample document is expected behaviour for a probabilistic transformer-based classifier and does not indicate a defect; it illustrates the importance of reporting a confidence score alongside every prediction so downstream users (clerks, judges) can calibrate trust in the AI suggestion. The 4-day average prediction error on the dashboard KPI, together with the 363–489-day interval on the Timeline Predictor, demonstrates that the system communicates uncertainty rather than presenting point estimates as ground truth — a design choice consistent with responsible-AI practice in a judicial context.

11. Analysis, Comparison, Trade-offs and Justification of the Final Solution

11.1 Design Alternatives Considered

- Monolithic server-rendered application (e.g., Django templates) vs. the chosen decoupled SPA + REST API: the SPA approach was selected to allow the front end and AI backend to evolve and scale independently, and to make the API reusable by a future mobile client.
- Client-side routing library (e.g., a JS framework such as React) vs. native <template> + vanilla JS: vanilla JS templates were chosen for the prototype to minimise build tooling and keep the deployment footprint (a single static folder served by Nginx) extremely small, at the cost of some scalability in state management as the UI grows.
- Synchronous vs. asynchronous AI inference: all AI calls (classification, prediction, redaction) are invoked asynchronously via fetch(), so a slow model inference does not block the rest of the dashboard — improving perceived responsiveness.

11.2 Trade-offs

- Simplicity vs. scalability: the current single-container backend and template-based frontend are easy to reason about and deploy, but would need a task queue and a component framework respectively to comfortably scale to many concurrent courts.
- Automation vs. oversight: AI classification, prediction and redaction accelerate registry work but are deliberately presented as assistive suggestions (with confidence scores and a "View" action for human review) rather than autonomous decisions, preserving human oversight over judicial data.
- Public transparency vs. privacy: the Citizen Portal increases public access to justice, but is paired with the Smart Redaction Engine so that any document surfaced publicly has PII/PHI stripped first — balancing the objectives of transparency and privacy protection.

11.3 Justification of the Final Solution

The chosen three-tier, containerised architecture was judged the best fit for an academic prototype that must be demonstrable end-to-end, be understandable by all team members regardless of frontend-framework experience, and be extensible enough to support the required AI modules without a heavyweight MLOps stack. The separation of the Dashboard (internal) from the Citizen Portal (public) directly reflects the differing security and information-disclosure requirements identified in the problem statement, and the explicit redaction step operationalises the privacy objective rather than leaving it as an unaddressed risk.

12. Broader Considerations / SDG Relevance

The project directly supports United Nations Sustainable Development Goal [SDG placeholder – typically SDG 16: Peace, Justice and Strong Institutions], which calls for effective, accountable and transparent institutions and equal access to justice for all. Specific alignments include:

- SDG 16 (Peace, Justice and Strong Institutions): the Citizen Portal promotes equal, low-friction public access to case status information, while AI-assisted workload analytics supports more accountable and efficient judicial administration.
- SDG 9 (Industry, Innovation and Infrastructure): the platform demonstrates responsible application of AI/NLP infrastructure to modernise a traditionally paper-based public institution.
- SDG 10 (Reduced Inequalities): automated redaction and a public tracking portal reduce the information and access asymmetry between well-resourced litigants (who can afford in-person registry visits or legal intermediaries) and the general public.

The team is mindful that AI-assisted classification and prediction outputs must remain advisory, auditable and subject to human review to avoid perpetuating bias in judicial administration — a consideration reflected in the UI's presentation of confidence scores and explicit "View" actions for every AI-generated suggestion.

13. Conclusion, Limitations and Possible Improvements

13.1 Conclusion

Smart Court AI demonstrates a working, containerised judicial case-management platform that integrates case administration, hearing scheduling, judicial analytics, AI-assisted document classification, timeline prediction, sensitive-data redaction and public case tracking within a single cohesive system. Manual testing across all major modules confirmed correct functional behaviour, and the architecture cleanly separates internal administrative tooling from public-facing services.

13.2 Limitations

- The AI classification and timeline-prediction modules are prototypes trained/demonstrated on a small, synthetic dataset and would require a much larger, jurisdiction-specific labelled corpus before production use.
- The redaction engine currently targets a fixed set of entity types (name, SSN, phone, email); more nuanced legal PII (e.g., case-specific identifiers, minors' information) would need additional entity classes.
- The frontend uses hard-coded API_BASE = 'http://localhost:8000/api', which is suitable for local development but must be externalised via environment configuration for multi-environment deployment.
- No authentication/authorisation layer is shown in the current UI; a production system would require role-based access control distinguishing judges, clerks, and citizens.
- The “precedent retrieval” objective is architected for but not yet implemented as a working module in this iteration.

13.3 Possible Improvements

- Introduce authentication (e.g., OAuth2/JWT) and role-based access control across the Dashboard, Case Manager and Citizen Portal.
- Replace the hard-coded API base URL with environment-driven configuration and add HTTPS termination at the Nginx layer.
- Expand the AI training corpus and add a human-in-the-loop feedback mechanism so clerks can correct misclassifications, improving the model over time.
- Implement the legal-precedent retrieval module using a vector-search index over historical judgments.
- Add automated integration/unit tests (e.g., pytest for the backend, Playwright for the frontend) to complement the manual test matrix in Section 8.

14. Individual Contribution of Group Members

Table 5 summarises each member's role and primary contributions to the project.

Reg. No. / Name	Role	Key Contributions
192511332 – Bheeshma L	Backend Lead	Designed and implemented the FastAPI backend and the /api/cases and /api/cases/hearings endpoints; coordinated the overall API contract used by the frontend.
192572182 – Ravuru Thrishank	Database Design Lead	Designed the PostgreSQL schema for cases, hearings, judges and status metadata; implemented query logic behind the Case Manager search and Pending Cases listing.
192524293 – Lella Thannveer Reddy	Frontend Lead	Built the single-page application shell (Dashboard, Analytics, Citizen Portal), the switchAppView()/switchTab() routing logic, and the visual design (glass-panel UI, status badges).
192525119 – Anneboina Sreenath	AI – Document Classification	Prototyped the LegalBERT-based document classification module and its /ai/classify-document endpoint integration; prepared sample legal filings used for testing.
192425148 – Chitturi Bhaskar Vinay	AI – Timeline Predictor	Prototyped the case-duration prediction logic and confidence-interval calculation; integrated the Timeline Predictor card and /ai/predict-timeline endpoint.
192465066 – P Soma Shankar Reddy	Security – Redaction Engine	Implemented the Smart Redaction Engine (PII/PHI detection for names, SSNs, phone numbers and emails) and the Redacted Vault UI/history view.
192365060 – P Sai Rakesh Reddy	DevOps / Deployment Lead	Authored the Docker Compose orchestration, backend Dockerfile and Nginx configuration; managed containerised deployment and environment setup.
192425080 – Shaik Arshad	Testing & Documentation Lead	Executed the end-to-end test-case matrix (Section 8), captured execution screenshots, and compiled and formatted the assignment report.

Table 5: Individual Contributions of Group Members

15. References

1. FastAPI Documentation. Available at: https://fastapi.tiangolo.com/ (accessed [02/09/2026]).
2. PostgreSQL Documentation. Available at: https://www.postgresql.org/docs/ (accessed [02/09/2026]).
3. Docker Compose Reference. Available at: https://docs.docker.com/compose/ (accessed [02/09/2026]).
4. Nginx Documentation. Available at: https://nginx.org/en/docs/ (accessed [02/09/2026]).
5. Feather Icons. Available at: https://feathericons.com/ (accessed [02/09/2026]).
6. Chalkidis, I. et al. "LEGAL-BERT: The Muppets straight out of Law School." Findings of EMNLP, 2020 (reference basis for the LegalBERT-style document classifier).
7. United Nations. "Sustainable Development Goal 16: Peace, Justice and Strong Institutions." Available at: https://sdgs.un.org/goals/goal16 (accessed [02/09/2026]).
8. Dataset: synthetic/sample case, hearing and document data generated in-house for demonstration purposes (sample_lawsuit.pdf and related mock records); no external proprietary dataset was used.
9. Course lecture notes and slides on Software Engineering, Database Systems, AI/NLP, Information Security and Cloud Computing, CSA1013 – SOFTWARE ENGINNERING, SAVEETHA SCHOOL OF ENGINEERING

Individual Reflections – All Team Members

Each member's one-page (condensed to half-page) individual reflection follows, describing design/development decisions, challenges faced, learning gained, the relevant SDG connection, and how the assignment contributed to the mapped Course Outcome(s).

192511332 – Bheeshma L

Design & Development Decisions: I owned the FastAPI backend and the /api/cases and /api/cases/hearings endpoints. I decided to keep the API contract simple and predictable (flat JSON case/hearing objects) so the frontend team could build against it early, before AI modules were finalised.

Challenges Faced: Agreeing on a stable API contract while the data model was still evolving was the hardest part — changes to a field name broke the frontend fetch calls more than once, which taught me to version and document the contract early.

Learning Gained: I learned how to design a REST API that is easy for a plain-JavaScript frontend to consume without a heavy client library, and how important consistent error responses are for graceful UI fallback.

SDG Connection: Building the case-retrieval backbone that powers both the internal dashboard and the public Citizen Portal reinforced SDG 16 (Peace, Justice and Strong Institutions) — reliable APIs are the invisible infrastructure behind equal access to justice information.

Contribution to CO: This work mapped to the mapped CO on designing and implementing a backend service for a real-world data-driven application, taking me through requirements-to-API design end to end.

192572182 – Ravuru Thrishank

Design & Development Decisions: I designed the PostgreSQL schema for cases, hearings, judges and status metadata. I chose normalized tables with a clear status enumeration (Pending Hearing, Under Review, Active, Discovery, Awaiting Filing) so the UI's colour-coded badges map directly to a single database field.

Challenges Faced: Balancing normalisation against query simplicity was tricky — an overly normalised schema made the Pending Cases and Case Manager queries slower to write, so I had to compromise with a few denormalised lookup fields (e.g., assigned_judge as text).

Learning Gained: I gained hands-on experience translating a real administrative workflow (court case lifecycle) into a relational schema, and understood why query-driven API design is preferable to exposing raw SQL to the client.

SDG Connection: A well-structured case database is foundational to SDG 16 — without accurate, structured records, neither workload analytics nor public case tracking would be trustworthy.

Contribution to CO: This contributed to the mapped CO on applying database-systems concepts to design a persistent data layer for a functioning software system.

192524293 – Lella Thannveer Reddy

Design & Development Decisions: I built the single-page application shell — the Dashboard, Analytics, and Citizen Portal views — along with the switchAppView()/switchTab() routing logic. I deliberately avoided a JS framework so every teammate could edit the UI directly using plain HTML templates.

Challenges Faced: Keeping view-state consistent (active tab highlighting, re-initialising Feather icons after every template swap) without a framework's reactivity required careful manual DOM management, which was easy to get subtly wrong.

Learning Gained: I learned how far you can get with template-driven vanilla JavaScript for a small SPA, and where a framework would start paying off as the UI grows in complexity.

SDG Connection: Designing separate, purpose-built views for internal staff and for the public Citizen Portal directly supports SDG 16 by giving ordinary citizens a dedicated, uncluttered way to access justice-system information.

Contribution to CO: This mapped to the CO on applying human–computer interaction and software-engineering principles to build a usable, modular front-end system.

192525119 – Anneboina Sreenath

Design & Development Decisions: I prototyped the LegalBERT-style document classification module and wired it to the /ai/classify-document endpoint. I chose to always surface the confidence score alongside the predicted category rather than a bare label, so clerks can judge how much to trust each prediction.

Challenges Faced: Getting consistent categories across different sample filings was difficult with a small demo dataset — the same document type sometimes produced different confidence levels across runs, which pushed me to treat the classifier strictly as advisory.

Learning Gained: I learned practical considerations in deploying an NLP classifier behind a REST endpoint, and the importance of communicating model uncertainty rather than hiding it.

SDG Connection: Automating legal-document triage supports SDG 16 by helping registries process filings faster and more consistently, freeing staff time for higher-value judicial work.

Contribution to CO: This contributed to the CO on applying AI/NLP concepts to solve a real classification problem within a larger software system.

192425148 – Chitturi Bhaskar Vinay

Design & Development Decisions: I built the case-duration prediction logic and its confidence-interval calculation, exposed through the Timeline Predictor card. I decided to always return a range (e.g., 363–489 days) instead of a single number, to avoid presenting a forecast as guaranteed fact.

Challenges Faced: Calibrating a believable confidence interval with very limited historical case data was the biggest challenge — too narrow an interval overstates certainty, too wide makes the prediction unhelpful.

Learning Gained: I learned how to reason about predictive uncertainty and communicate it responsibly in a UI, a skill that goes beyond just calling a model and printing its output.

SDG Connection: Accurate scheduling forecasts support SDG 16 by helping courts plan hearings and manage litigant expectations, reducing needless delay in the justice process.

Contribution to CO: This mapped to the CO on applying predictive-modelling concepts to a socially relevant, data-driven problem.

192465066 – P Soma Shankar Reddy

Design & Development Decisions: I implemented the Smart Redaction Engine that detects and masks names, SSNs, phone numbers and emails, and built the Redacted Vault UI. I chose token-style placeholders (e.g., [REDACTED_SSN]) instead of simple blanking, so reviewers can still see what type of data was removed.

Challenges Faced: Reliably detecting all PII patterns (especially names) without over-redacting ordinary text was harder than expected, and taught me the limits of pattern-based detection compared to a trained NER model.

Learning Gained: I gained a much deeper appreciation of privacy engineering — the difference between deleting data and truly making it non-identifiable, and why redaction has to happen before any public release, not after.

SDG Connection: This module operationalises the privacy side of SDG 16, ensuring greater public transparency does not come at the cost of exposing litigants' personal information.

Contribution to CO: This work mapped to the CO on applying information-security and privacy-engineering concepts within a real software system.

192365060 – P Sai Rakesh Reddy

Design & Development Decisions: I authored the docker-compose.yml, the backend Dockerfile, and the Nginx configuration serving the static frontend. I chose to containerise the backend and frontend as two independent services so either could be redeployed or scaled without touching the other.

Challenges Faced: Getting the Nginx container to correctly serve the frontend while the backend ran on a separate port required trial and error with volume mounts and port mapping, especially while keeping the setup reproducible for every team member's machine.

Learning Gained: I learned practical container orchestration — service dependencies (depends_on), restart policies, and read-only volume mounts — and how these choices affect the reliability of a deployed system.

SDG Connection: A cleanly containerised, reproducible deployment lowers the barrier for smaller or under-resourced courts to adopt this kind of tool, which speaks to SDG 9 (Innovation and Infrastructure) in support of SDG 16's justice-access goals.

Contribution to CO: This contributed to the CO on applying cloud-computing/DevOps concepts to deploy a multi-service application.

192425080 – Shaik Arshad

Design & Development Decisions: I owned end-to-end testing and the assignment report. I decided to build a structured test-case matrix (Section 8) covering every module rather than ad-hoc spot checks, so the whole team could verify functionality consistently before each demo.

Challenges Faced: Reproducing certain states (e.g., a deliberately stopped backend to test failure handling) without disrupting teammates' active development was a coordination challenge, which we solved by testing against a separate local instance.

Learning Gained: I learned how to translate a working application into a clear, evidence-backed report — pairing every claim with a concrete screenshot or test result rather than an unverified statement.

SDG Connection: Rigorous testing and transparent documentation of an access-to-justice tool is itself an application of SDG 16's call for accountable institutions — a system this important should be demonstrably reliable, not just functional.

Contribution to CO: This mapped to the CO on validating a software system through systematic testing and communicating results in a professional engineering report.